

Discussion 3

—
Jett Rosen - Sections 13 and 14

Computing power: *exists*

Recursive Fibonacci:



All feelings are temporary, you are loved,
you are enough, and you matter.

Logistics

- Attendance QR code will be on the last slide, make sure to fill it out!
- HW3 is out
 - Start early!

Divide and Conquer vs. Dynamic Programming

Divide and Conquer	Dynamic Programming
Divide original problem to smaller version(s) of the same problem	
Non-overlapping subproblems	Overlapping subproblems
Optimal substructure: The solution is correct for this (scaled-down) portion	Optimal substructure: The solution is correct up to this point
Always top-town	Top-down, memoization, bottom-up
Often less time efficient	Often more time efficient- especially with bottom-up

Dynamic Programming

- Break problem down into **overlapping** subproblems, and store the results
 - We essentially trade memory complexity for improved runtime complexity
- DP requires:
 - **Optimal Substructure:** The “optimal” result is some combination of “optimal” subproblems
 - **Overlapping Subproblems:** The same inputs occur many times when recursively solving a problem

Dynamic Programming

- Most of the creativity for a DP problem comes from creating the recurrence:
 1. Define your variables
 - a. Often an index, in 2d problems it can be beneficial to consider the first i elements
 2. Define your base case(s)
 3. Write out problem in terms of subproblems
 - a. Usually multiple cases
- Runtime analysis is somewhat mechanical:
 - Runtime is the time it takes to fill one cell of the table times the size of the table

Recurrence Relations: 3-step Recipe

Before thinking about the implementation, first formulate the recurrence relation mathematically. Here's a 3-step recipe for this task:

- Step 1: Define the subject of recurrence (in English)
- Step 2: Identify the base case(s)
- Step 3: Identify subproblem dependencies

Dynamic Programming

- In general, there are some “types” of DP problems:
 - Many problems that seem different at first glance may be solved by similar recurrences

When the interview is going really well and the interviewer asks you a dyanmic programming problem



When the interview is going really well and the interviewer asks you a dyanmic programming problem



Conquering the Fear of DP

- **You're Not Alone!** Many of us find dynamic programming challenging at first
- **Understand the Problem:** Regularly practice **reading** prompts to grasp the problem's requirements
 - Check out [geeksforgeeks.org](https://www.geeksforgeeks.org) for list of DP problems- getting comfortable reading the prompt makes you confident
- **Visualizers:** Explore visualizers to understand the shape and structure of the DP table
 - Check out [Algorithm visualizer](#)
- **Break It Down:** Tackle each component of dynamic programming **separately** to avoid feeling overwhelmed:
 - Dimensionality
 - Recurrence relation
 - Implementation
 - Reconstructing solution
- **Ask Us for Help!** We are more than happy to talk about **anything** you're struggling with!

Useful Notations

Sum Σ / product Π

Over an array
 $A[1, \dots, n]$ $\sum_{i=1}^n A[i] = A[1] + A[2] + \dots + A[n]$

$$\prod_{i=1}^n A[i] = A[1] \cdot A[2] \cdot \dots \cdot A[n]$$

Over a set
 $S = \{s_1, \dots, s_n\}$ $\sum_{s \in S} s$ $\prod_s s$

Conditional $\sum_{i: i \text{ is even}} A[i]$ $\prod_{s \in S: s > 0} s$

Max/ min

Over an array
 $A[1, \dots, n]$ $\max_{i: 1 \leq i \leq n} A[i]$ Over a set
 $S = \{s_1, \dots, s_n\}$ $\min_{s \in S} s$

Sets Operations

Union $\bigcup_{i=1}^n S_i = S_1 \cup S_2 \cup \dots \cup S_n$

Intersection $\bigcap_{i=1}^n S_i = S_1 \cap S_2 \cap \dots \cap S_n$

Set minus $A \setminus B = A \cap \bar{B}$

It is common to write $S \leftarrow S \setminus \{s\}$ for
"remove element s from set S "

Big AND $\wedge$ / big OR $\vee$

Over an array
of truth values
 $T[1, \dots, n]$ $\bigwedge_{i=1}^n T[i] = T[1] \wedge T[2] \wedge \dots \wedge T[n]$

$$\bigvee_{i=1}^n T[i] = T[1] \vee T[2] \vee \dots \vee T[n]$$

Exit Ticket

